

RGPVMate

AI-Powered University Assistant for RGPV Students

Project Technical Document · RAG Architecture

Himanshu Lokhande

CSE 2027

Technocrats Institute of Technology

RGPV · Bhopal

1. Project Overview

RGPVMate is a **RAG-based (Retrieval Augmented Generation)** AI chatbot designed specifically for students affiliated with Rajiv Gandhi Proudhyogiki Vishwavidyalaya (RGPV), Bhopal. Students can ask any question related to their university — syllabus, previous year papers, examination rules, grading schemes, academic calendar, and more — and receive accurate, sourced answers instantly in plain English or Hinglish.

The core problem RGPVMate solves is **information fragmentation**. A student looking for basic university information must currently navigate the RGPV website, dig through WhatsApp groups, ask seniors, or search through poorly organised PDFs. RGPVMate consolidates all of this into a single conversational interface.

- Unlike simply calling a general-purpose AI — which may hallucinate or lack RGPV-specific knowledge — RGPVMate grounds the AI's responses in real RGPV documents. The AI can only answer from what is fed into the system, eliminating hallucination for domain-specific queries.

2. The Problem Being Solved

Every RGPV student regularly faces these friction points:

- **Syllabus confusion:** Students are unsure which topics fall under which unit, especially after scheme changes (2019 vs 2022 scheme).
- **PYQ access:** Previous year question papers are scattered across unofficial student websites, often incomplete or incorrectly labelled.
- **Exam rule ambiguity:** Attendance requirements, backlog policies, and condonation rules are buried in lengthy ordinance PDFs.
- **Academic calendar:** Exam dates require navigating an often-outdated university website.
- **General LLM failure:** Asking Gemini or ChatGPT about RGPV-specific information results in hallucinated answers — RGPV data is minimally represented in their training data.

3. What is RAG — The Core Technology

RAG stands for Retrieval Augmented Generation. It combines two things: **Retrieval** — finding the most relevant pieces of information from a document database — and **Generation** — using an LLM to read those pieces and produce a clean, natural language answer.

- Open-book analogy: A normal LLM is like a student answering from memory — it may remember wrong things. A RAG system is like an open-book exam — the student reads the right pages and answers from what is written there. The LLM in RGPVMate is the reading and formatting engine. The knowledge lives in your documents.

4. System Architecture

RGPVMate operates across **two Render services** and three data stores. The architecture has two distinct operational phases.

Service	Technology	Render Instance	Responsibility
Service 1 — Main Backend	Node.js + Express	Free Web Service	API routes, orchestration, auth, admin
Service 2 — Embedder	Python + Flask	Free Web Service	Sentence Transformers, text vector

4.1 Offline Ingestion Pipeline (Runs Once)

Processes all university PDFs and loads them into ChromaDB. Triggered manually via the Admin Panel or ingestion script.

```
PDF Documents (Syllabus, PYQs, Rules, Calendar)
  |
  v
pdf-parse → extracts raw text from each PDF
  |
  v
chunker.js → YOUR logic splits text into meaningful units
            different strategy per document type
            attaches metadata (semester, subject, type, scheme)
  |
  v
Node.js calls Python Embedder Service (HTTP POST /embed)
Flask microservice runs model.encode(chunk_text)
→ returns 384-dimensional vector [0.23, 0.87, 0.12 ...]
→ no internet needed | no API cost | runs on CPU
  |
  v
ChromaDB stores: chunk text + vector + metadata
(PDFs are NOT stored in ChromaDB – only text + vectors)
  |
  v
Done. Knowledge base is ready. Server never needs restart.
```

4.2 Live Query Pipeline (Runs Per Student Request)

```
Student types: 'What is the attendance rule for RGPV?'
|
v
React Frontend → HTTP POST to Node.js backend
|
v
Node.js → HTTP POST to Python Embedder Service
          { text: 'What is the attendance rule?' }
          Embedder returns vector [0.31, 0.79, 0.18 ...]
|
v
ChromaDB similarity search → finds top 4 matching chunks
(compares question vector against all stored chunk vectors)
|
v
Node.js builds structured prompt:
[ System Guardrail + 4 Relevant Chunks + Student Question ]
|
v
Gemini 1.5 Flash → ONLY external API call remaining
reads chunks → generates grounded, accurate answer
|
v
If student is logged in → save Q&A to MongoDB (background)
|
v
Student sees: Answer + Source document name + Suggestions
```

5. Chunking Strategy

Chunking splits large PDFs into smaller, meaningful units before storing them. **This is the most critical design decision** — poor chunking means wrong chunks are retrieved and wrong answers are generated. RGPVMate uses a different chunking strategy per document type because each has a different natural structure.

Document Type	Chunking Strategy	Why This Boundary
Syllabus	Split by Unit per Subject	Each unit is self-contained — clean semantic boundary
PYQ Papers	Split by Question Number	Each question is independent — enables precise retrieval
Exam Rules	Split by Clause / Paragraph	Each rule is independent — prevents mixing regulations

Academic Calendar	Split by Event / Date Block	Each date entry is standalone
Fee Structure	Split by Fee Category	Different fee types retrieved independently

Metadata Attached to Every Chunk

Metadata enables filtered retrieval — searching only semester 5 chunks for a semester 5 student:

```
{
  text:      'CS-501 Computer Networks — Unit 3: Network Layer,
             IP Addressing, Routing Algorithms, OSPF...',
  vector:    [0.23, 0.87, 0.12, ...]  // 384 numbers
  metadata: {
    source:   'RGPV_CSE_Syllabus_2022.pdf',
    type:     'syllabus',
    semester: 5,
    subject:  'Computer Networks',
    unit:     3,
    scheme:   '2022'
  }
}
```

- What ChromaDB actually stores: chunk text + vector + metadata. The original PDF file is NOT stored in ChromaDB — it stays in your /documents folder on the server. ChromaDB only needs text and vector to do its job.

6. Embeddings — Sentence Transformers vs Gemini

The original design used the Gemini Embedding API to convert text to vectors. This was replaced with **Sentence Transformers running locally** to eliminate external API dependency at the most critical pipeline step.

	Gemini Embedding API	Sentence Transformers (chosen)
Model runs on	Google's servers	Your own server (CPU)
Vector dimensions	768	384
Internet required	Yes — every request	No — after initial download
Cost	Free tier with rate limits	Completely free, no limits

Speed	200–400ms (network)	20–50ms (local CPU)
English accuracy	Excellent	Excellent
Hinglish accuracy	Very good	Good (minor gap)
Model size	N/A (remote)	~90MB downloaded once
Single point fail	Yes — Google API outage	No — fully self-contained

How the Python Embedder Service Works

```
# embed_service.py – runs as separate Flask server on port 5000
from flask import Flask, request, jsonify
from sentence_transformers import SentenceTransformer

app = Flask(__name__)
model = SentenceTransformer('all-MiniLM-L6-v2')
# Model loads ONCE when server starts – stays in memory permanently
# 90MB download on first run, then cached forever

@app.route('/embed', methods=['POST'])
def embed():
    text = request.json['text']
    vector = model.encode(text).tolist()
    return jsonify({ 'vector': vector })
    # Returns in ~20-50ms – pure CPU math, no network call

app.run(port=5000)
```

How Node.js Calls The Python Service

```
// retriever.js – Node.js calls Python over HTTP
async function getEmbedding(text) {
    const response = await fetch(process.env.EMBEDDER_URL + '/embed', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text })
    })
    const data = await response.json()
    return data.vector // 384-dim array
}

// EMBEDDER_URL = 'http://localhost:5000' (development)
// EMBEDDER_URL = 'https://rgpvmate-embedder.onrender.com' (production)
```

7. Prompt Construction — Preventing Hallucination

The most important line in the entire project is the system instruction that prevents Gemini from answering using its own training data. This is called **prompt guardrail**.

```
SYSTEM PROMPT sent to Gemini with every request:
██████████████████████████████████████████████████████████████████████████

You are RGPVMate, an AI assistant for RGPV students.
Answer ONLY using the context provided below.
If the answer is not in the context, say:
    'I do not have that information in my knowledge base.'
Always cite the source document at the end of your answer.
Answer naturally in English or Hinglish based on how the student asks.
```

```
CONTEXT (top 4 chunks retrieved from ChromaDB):
■■ Chunk 1   (RGPV_Examination_Ordinance_2023.pdf) ██████████
Minimum 75% attendance required for semester exams...

■■ Chunk 2   (RGPV_CSE_Syllabus_2022.pdf) ████████████████████████████████
CS-303 Data Structures – Unit 1: Arrays, Linked Lists...

STUDENT QUESTION:
Kitne attendance chahiye exam dene ke liye?
```

- Without this guardrail: Gemini answers from training data may hallucinate RGPV rules. With this guardrail: Gemini reads ONLY your retrieved chunks grounded, accurate answer. This is what separates RGPVMate from simply calling the Gemini API directly.

8. Deployment — Two Render Services

Both services deploy on Render's free tier under the same account. They communicate internally over HTTPS. The URL switches automatically between local and production via environment variables — no code changes needed.


```
Render Account (free)
■■■ Service 1: rgpvmate-backend (Node.js) port 3000
■ URL: https://rgpvmate-backend.onrender.com
■ Handles: student chat, auth, admin, MongoDB, ChromaDB
■
■■■ Service 2: rgpvmate-embedder (Python) port 5000
URL: https://rgpvmate-embedder.onrender.com
Handles: text → vector conversion only
Model loaded once at startup, stays in memory

Vercel (free)
■■■ React Frontend
URL: https://rgpvmate.vercel.app
Calls: rgpvmate-backend.onrender.com/api/*
```

- Adding a new PDF does NOT require a server restart. ChromaDB is a separate database — adding chunks is a write operation. The server keeps running and serving students uninterrupted while you ingest new documents via the Admin Panel.

9. Admin Panel – Document Management

The Admin Panel is a password-protected section of the app (only accessible to you). It handles all document lifecycle operations without touching the codebase. Authentication uses a hardcoded password stored in Render environment variables.

9.1 What The Admin Panel Shows

Admin Panel (password protected)

■

Current Documents in ChromaDB

■

■

■

■

RGPV_CSE_Syllabus_2022.pdf

234 chunks

[Replace]

[Delete]

■

■

RGPV_Ordinance_2023.pdf

89 chunks

[Replace]

[Delete]

■

■

RGPV_PYQ_CS501_2023.pdf

42 chunks

[Replace]

[Delete]

■

■

RGPV_Calendar_2024.pdf

28 chunks

[Replace]

[Delete]

■

■

■

■

Upload New Document

■

■

[Choose PDF]

[Document Type ▼]

■

■

[Ingest Document]

■

9.2 Replace Flow — No Manual Chunk Selection

You always replace at the **document level** — never individual chunks. When you click Replace on a document, the system automatically handles everything:

```
You click Replace on 'RGPV_Ordinance_2023.pdf'
|
v
Admin panel asks you to upload the new PDF
|
v
Step 1 – Delete all old chunks
chromaDB.delete({ where: { source: 'RGPV_Ordinance_2023.pdf' } })
→ removes all 89 chunks from old document automatically
|
v
Step 2 – Process new PDF
pdf-parse extracts text → chunker.js creates chunks
→ Node.js calls Python Embedder for each chunk
|
v
Step 3 – Store new chunks
chromaDB.add(newChunks with source: 'RGPV_Ordinance_2024.pdf')
|
v
Admin panel: '■ Replaced – 94 new chunks stored successfully'
Bot immediately answers from the new document.
```

10. Authentication & User Data

10.1 Two-Tier Access Model

RGPVMate uses optional authentication to balance open access with personalisation:

- **No login required:** Any RGPV student opens the app and starts chatting instantly. Zero friction, zero signup. Nothing stored about them.
- **Optional Google OAuth:** Students who sign in get persistent chat history and saved branch/semester preferences.
- **Admin access:** Password-protected via environment variable. No OAuth — just a secret key in request headers.

10.2 Google OAuth Flow

```
Student clicks 'Sign in with Google'
  |
  v
App redirects to Google login page
(you never see their password – Google handles everything)
  |
  v
Student approves – Google sends profile to your backend:
{ googleId: '117382910234', email: 'him@gmail.com',
  name: 'Himanshu', picture: 'https://...' }
  |
  v
Backend creates JWT token – sends to frontend
Frontend stores JWT in memory – sends with every request
  |
  v
First login: user document created in MongoDB
Return visit: existing user found by googleId
```

10.3 What Gets Stored In MongoDB

Two collections only. Everything is minimal – only what is needed for personalisation:

```
// Collection 1: users
{
  _id:      ObjectId(),
  googleId: '117382910234', // unique ID from Google
  email:    'him@gmail.com', // display only
  name:     'Himanshu',      // personalised greeting
  picture:  'https://...',   // profile photo
  branch:   'CSE',           // student preference
  semester: 5,               // student preference
  createdAt: ISODate(...)

  // NEVER stored: password, Google access token, sensitive data
}

// Collection 2: chat_history (logged-in students only)
{
  _id:      ObjectId(),
  userId:   '117382910234',
  question: 'What is the attendance rule?',
  answer:   'RGPV requires minimum 75%..',
  sources:  ['RGPV_Ordinance_2023.pdf'],
  semester: 5,
  branch:   'CSE',
  createdAt: ISODate(...)
}

// Cap: last 50 conversations per user – older ones auto-deleted
```

11. Document Versioning Strategy

Document	Change Frequency	Update Action
Syllabus	Rarely – every 4–5 years	Reingest when new scheme released
PYQ Papers	Never changes, only grows	Add new year's paper each year
Exam Rules / Ordinance	Occasionally – ~once/year	Delete old + reingest new via Admin Panel
Academic Calendar	Every academic year	Update at semester start
Fee Structure	Every academic year	Update at semester start

12. Project File Structure

```
rgpvmate/
├── backend/                                (Node.js – Render Service 1)
│   ├── scripts/
│   │   ├── ingest.js                      Run once: loads all PDFs into ChromaDB
│   │   └── update.js                      Run when a document changes
│   ├── services/
│   │   ├── chunker.js                     Chunking logic per document type
│   │   ├── retriever.js                   Calls Python embedder + searches ChromaDB
│   │   └── llm.js                         Builds prompt + calls Gemini Flash
│   ├── routes/
│   │   ├── chat.js                        POST /api/chat – handles student queries
│   │   ├── auth.js                       Google OAuth flow + JWT
│   │   ├── history.js                     GET/POST /api/history – chat history
│   │   └── admin.js                       Admin routes – document management
│   ├── middleware/
│   │   ├── authGuard.js                  Verifies JWT for protected routes
│   │   └── adminGuard.js                 Verifies admin password header
│   └── index.js                           Express server entry point
├── embedder/                              (Python – Render Service 2)
│   ├── embed_service.py                   Flask server, Sentence Transformers
│   └── requirements.txt                   flask, sentence-transformers, torch
├── documents/                             (PDF storage on server)
│   ├── syllabus/
│   ├── pyq/
│   ├── rules/
│   └── calendar/
└── frontend/                              (React – Vercel)
    ├── src/
    │   ├── components/
    │   │   ├── ChatWindow.jsx
    │   │   ├── MessageBubble.jsx
    │   │   ├── SuggestedQuestions.jsx
    │   │   ├── SemesterSelector.jsx
    │   │   ├── ChatHistory.jsx
    │   │   └── AdminPanel.jsx
    │   ├── App.jsx
    │   └── main.jsx
```

13. Estimated System Scale

Metric	Estimate	Notes
--------	----------	-------

Total PDFs	~50	Syllabus, PYQs, rules, calendar, misc
Total chunks in ChromaDB	~900–1,100	Varies by document length
Avg chunk size	300–500 words	Tuned per document type
Chunks retrieved per query	4	Top-4 by cosine similarity
Embedding dimensions	384	Sentence Transformers output
ChromaDB search time	< 100ms	1,100 vectors is trivial
Embedder response time	20–50ms	Local CPU, model in memory
Total query response time	1.5–3 sec	Embed + retrieve + Gemini generation
MongoDB collections	2	users + chat_history
Max chat history per user	50 messages	Older entries auto-deleted

14. Complete Tech Stack — What and Why

React + Tailwind CSS

Role: Frontend UI

React enables a dynamic component-based chat interface where messages update in real time without full page reloads. Tailwind CSS provides utility-first styling for rapid, responsive UI development. React's state management handles accumulating chat messages, loading states, and semester filter application to queries dynamically.

Node.js + Express

Role: Main Backend (Service 1)

Node.js keeps the entire stack in JavaScript, reducing context switching. Express provides a minimal framework for API routes. The backend orchestrates everything: receiving queries, calling the Python embedder, searching ChromaDB, calling Gemini, handling auth, and managing the admin panel. No Python required for this service.

Python + Flask

Role: Embedding Microservice (Service 2)

A minimal Flask server with one job: receive text, return a vector. Flask was chosen because Sentence Transformers is a Python library — the microservice pattern keeps the Python footprint small and isolated. Node.js calls it over internal HTTP.

Sentence Transformers

Role: Text Embeddings (Local)

Runs the all-MiniLM-L6-v2 model locally on the server CPU. Downloads 90MB once on first run, then stays in memory. Zero API calls, zero cost, zero rate limits. Replaces Gemini Embedding API to eliminate external dependency at the most critical pipeline step. Quality gap vs Gemini is negligible for RGPV document retrieval.

ChromaDB

Role: Vector Database

Open-source vector database built for AI applications. Stores chunk text, 384-dimensional vectors, and metadata. Performs cosine similarity search to find the most semantically relevant chunks for any query. Chosen over Pinecone because it is fully open-source, self-hostable at zero cost on Render, and has a native JavaScript client.

Gemini 1.5 Flash

Role: Answer Generation

The only remaining external API dependency. Used solely for generating clean, natural language answers from retrieved chunks. Flash is faster and cheaper than Pro while being fully capable for short-context generation tasks. Free tier is generous enough for a student project with moderate traffic.

MongoDB

Role: User Data & Chat History

Stores two collections: users (googleId, name, branch, semester preference) and chat_history (last 50 Q&A; pairs per logged-in user). Chosen because it fits naturally with a Node.js stack and the flexible document structure suits user preference storage.

Google OAuth 2.0 + JWT

Role: Optional Authentication

Google OAuth handles login — you never store passwords. After OAuth, a JWT token is issued for session management. Authentication is optional — unauthenticated students can use the full chat without signing in. Only signed-in users get persistent history.

Vercel + Render

Role: Deployment

React frontend on Vercel for instant global CDN with zero configuration. Both Node.js and Python services on Render's free tier. Together they provide a fully deployed, publicly accessible application at zero infrastructure cost.

Technology	Purpose	Why Chosen
React + Tailwind	Frontend UI	Dynamic chat, responsive, rapid development
Node.js + Express	Main Backend	JS consistency, orchestration hub
Python + Flask	Embedding Service	Sentence Transformers is Python-native
Sentence Transformers	Text Vector	Free, local, no rate limits, 90MB model
ChromaDB	Vector Database	Open-source, free hosting, JS client
Gemini 1.5 Flash	Answer Generation	Fast, free tier, only external call
MongoDB	Users + Chat History	Flexible schema, Node.js native
Google OAuth 2.0 + JWT	Optional Auth	No password storage, trusted provider
Vercel + Render	Deployment	Free tier covers entire project

15. Key Interview Talking Points

Read these answers aloud before your placements. These are the most likely technical questions.

Q: What is RAG and why did you use it?

RAG stands for Retrieval Augmented Generation. Instead of relying on what an LLM was trained on, I feed it relevant documents at query time. I used it because RGPV-specific information is not well represented in general LLM training data — asking Gemini directly about RGPV would result in hallucinated answers. RAG grounds the model in real documents I control.

Q: How did you handle chunking?

I wrote different chunking strategies per document type. For the syllabus I split by unit headings since each unit is a natural semantic boundary. For PYQs I split by question numbers. I also attached metadata like semester and subject so I could do filtered retrieval — only searching 5th semester chunks when a 5th semester student asks a question.

Q: How do you prevent hallucination?

The system prompt explicitly instructs Gemini to answer only from the provided context and to say it does not know if the information is unavailable. This is called prompt guardrailing. The LLM cannot fill gaps from its training data — it reads only the retrieved chunks from my documents.

Q: Why did you separate Node.js and Python into two services?

Sentence Transformers — the library I use for text embeddings — is Python-native and cannot run in Node.js. I built a minimal Flask microservice that handles only embedding requests. Node.js calls it over internal HTTP. This keeps the Python footprint small and isolated while the rest of the stack stays in JavaScript.

Q: Why Sentence Transformers instead of Gemini Embedding API?

Gemini Embedding has rate limits and creates an external dependency at the most critical step of my pipeline. Sentence Transformers runs locally on my server CPU — no API calls, no cost, no rate limits. The model downloads once and stays in memory. For RGPV technical text, the quality difference is negligible.

Q: What happens when university documents change?

I built a versioned ingestion system via an admin panel. Each chunk carries the source filename as metadata. When RGPV updates a document, I click Replace in the admin panel, upload the new PDF, and the system automatically deletes all old chunks from that source and reingests the new version. No server restart needed.

Q: How is user data protected?

Google OAuth means I never store passwords — Google handles authentication and only sends me a user's public profile after they explicitly approve. I store only what is necessary for personalisation: googleId, name, and branch/semester preference. Unauthenticated users have zero data stored about them.

16. Summary

RGPVMate is not just an API call to Gemini with a chat UI. It is a complete, production-grade AI pipeline — two deployed services, intelligent document chunking, local vector embeddings, semantic retrieval, prompt guardrailing, optional authentication, persistent chat history, and an admin panel for document lifecycle management — all designed around a real problem faced by thousands of RGPV students.

- The system demonstrates: RAG architecture, microservice design, vector databases, semantic search, prompt engineering, document versioning, OAuth authentication, and eliminating third-party dependencies through local model hosting. Every design decision has a reason — and that reasoning is what makes this project stand out.

— End of Document —